



Week2 ZKP

NITESH

Sum check Protocol

- IP protocol where the prover proves to the verifier that a sum of large number of terms is correct

$$H := \sum_{b_1 \in \{0,1\}} \sum \dots \sum_{b_v \in \{0,1\}} g(b_1, b_2, \dots, b_n)$$

- It helps the verifier to check sum without calculating the whole sum.

Number of evaluations in the sum: 2^v

Time to compute each evaluation: T

Total time to compute the sum: $2^v \cdot T$

- At the start of the protocol, the prover sends a value C_1 claimed to equal the value H defined in Equation (4.1).
- In the first round, $\mathcal{P}$ sends the univariate polynomial $g_1(X_1)$ claimed to equal

$$\sum_{(x_2, \dots, x_v) \in \{0,1\}^{v-1}} g(X_1, x_2, \dots, x_v).$$

$\mathcal{V}$ checks that

$$C_1 = g_1(0) + g_1(1),$$

and that g_1 is a univariate polynomial of degree at most $\deg_1(g)$, rejecting if not. Here, $\deg_j(g)$ denotes the degree of $g(X_1, \dots, X_v)$ in variable X_j .

- $\mathcal{V}$ chooses a random element $r_1 \in \mathbb{F}$, and sends r_1 to $\mathcal{P}$.
- In the j th round, for $1 < j < v$, $\mathcal{P}$ sends to $\mathcal{V}$ a univariate polynomial $g_j(X_j)$ claimed to equal

$$\sum_{(x_{j+1}, \dots, x_v) \in \{0,1\}^{v-j}} g(r_1, \dots, r_{j-1}, X_j, x_{j+1}, \dots, x_v).$$

$\mathcal{V}$ checks that g_j is a univariate polynomial of degree at most $\deg_j(g)$, and that $g_{j-1}(r_{j-1}) = g_j(0) + g_j(1)$, rejecting if not.

- $\mathcal{V}$ chooses a random element $r_j \in \mathbb{F}$, and sends r_j to $\mathcal{P}$.
- In Round v , $\mathcal{P}$ sends to $\mathcal{V}$ a univariate polynomial $g_v(X_v)$ claimed to equal

$$g(r_1, \dots, r_{v-1}, X_v).$$

$\mathcal{V}$ checks that g_v is a univariate polynomial of degree at most $\deg_v(g)$, rejecting if not, and also checks that $g_{v-1}(r_{v-1}) = g_v(0) + g_v(1)$.

- $\mathcal{V}$ chooses a random element $r_v \in \mathbb{F}$ and evaluates $g(r_1, \dots, r_v)$ with a single oracle query to g . $\mathcal{V}$ checks that $g_v(r_v) = g(r_1, \dots, r_v)$, rejecting if not.
- If $\mathcal{V}$ has not yet rejected, $\mathcal{V}$ halts and accepts.

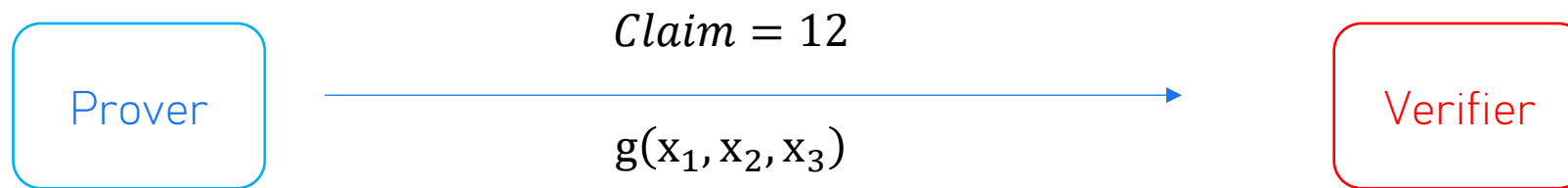
Example

- Suppose the prover want to show that the sum $H = 12$

$$\sum_{b_1, b_2, b_3 \in \{0,1\}} g(b_1, b_2, b_3) = 12 \text{ where } g(x_1, x_2, x_3) = x_1 + x_2 + x_3$$

Calculating it directly we get:

$$g(0,0,0) + g(0,0,1) + g(0,1,0) + g(0,1,1) + g(1,0,0) + g(1,0,1) + g(1,1,0) + g(1,1,1) = 12$$

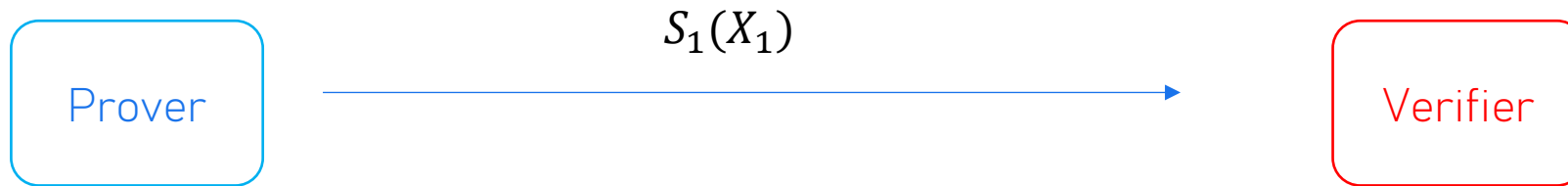


Round 1

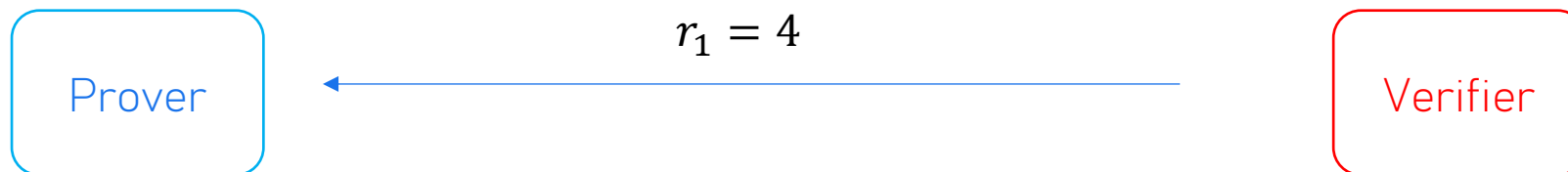
Prover sends a polynomial $S_1(X_1)$

$$S_1(X_1) = \sum_{b_2, b_3 \in \{0,1\}} g(X_1, b_2, b_3)$$

$$S_1(X_1) = g(X_1, 0,0) + g(X_1, 0,1) + g(X_1, 1,0) + g(X_1, 1,1) = 4X_1 + 4$$



The verifier checks whether if $S_1(0) + S_1(1) = 12 = H$, if it passes then sends a random value r_1

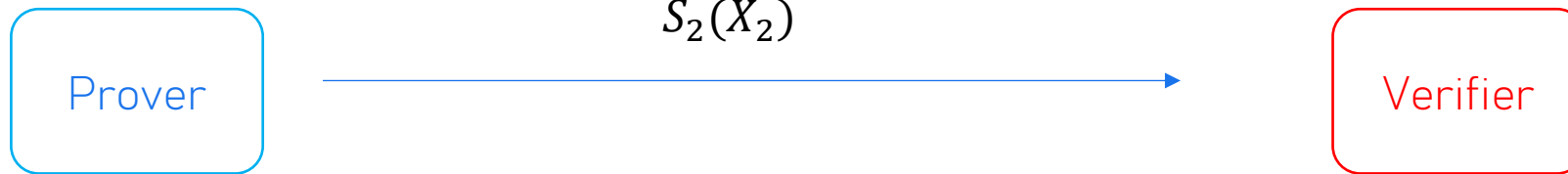


Round 2

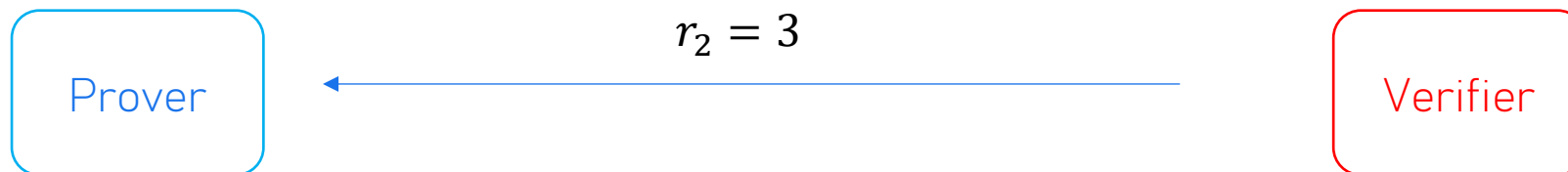
Prover sends a polynomial $S_2(X_2)$ where now the first variable is fixed to $r_1 = 4$

$$S_2(X_2) = \sum_{b_3 \in \{0,1\}} g(r_1, X_2, b_3)$$

$$S_2(X_2) = g(r_1, X_2, 0) + g(r_1, X_2, 1) = 2X_2 + 9$$



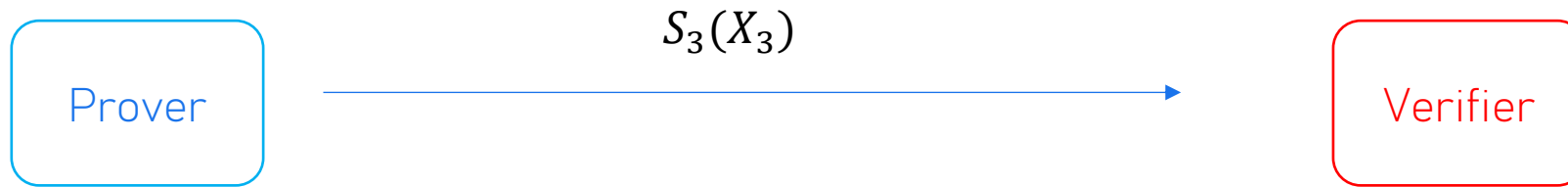
The verifier checks whether if $S_2(0) + S_2(1) = S_1(4) = 20$, if it passes then sends a random value r_2



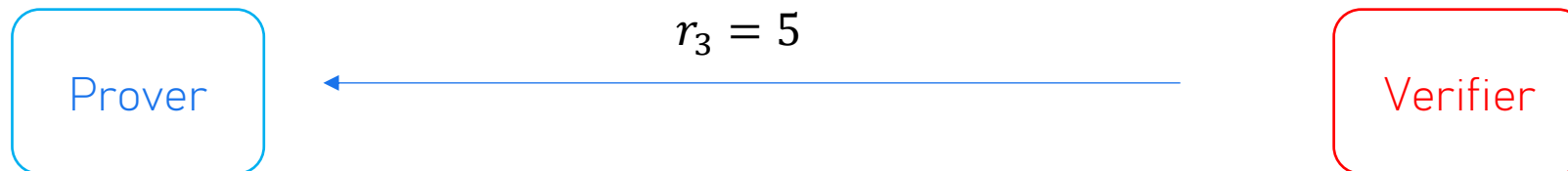
Round 3

Prover sends a polynomial $S_3(X_3)$ where now the second term is fixed to r_2

$$S_3(X_3) = g(r_1, r_2, X_3) = X_3 + 7$$



The verifier checks whether if $S_3(0) + S_3(1) = S_2(3) = 15$, if it passes then sends a random value $r_3 = 5$



Since it is the last round the verifier checks if $S_3(r_3) = g(r_1, r_2, r_3) = 12$, then the verifier accepts.

Time Complexity

Direct Computation: $O(2^v \cdot T)$ where T is the time taken to evaluate $g(x_1, x_2, x_3)$

Using Sum-Check:

Number of rounds: v

Verifier time = $O(v + \sum_{i=1}^V \deg_i(g) + T)$

For multilinear polynomials $\deg_i(g) = 1$ it becomes $O(v + T)$

Prover time: $O(\sum_{i=1}^V \deg_i(g) \cdot 2^{v-i} \cdot T)$

This is due to the prover need to compute $S_1(X_1), S_2(X_2) \dots, S_v(X_v)$.

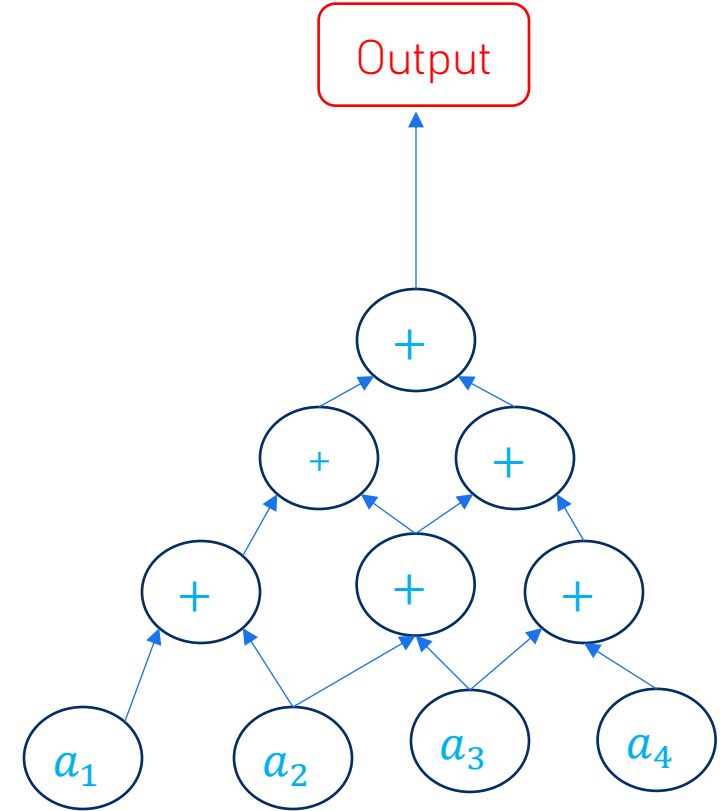
Approximately it becomes $O(2^v \cdot T)$

GKR Protocol

- An IP protocol used to verify the output of a computation represented as an arithmetic circuit.
- Verifier does not need to recompute the entire circuit
- Each circuit is arranged in layers $0, 1 \dots, d$
- Instead, the verifier checks the circuit layer by layer using the sum-check protocol
- It checks the value of the layer W_i using the gate values in the next layer W_{i+1}

$$W_i(g) = \sum_{a,b \in \{0,1\}^k} \text{add}_i(g, a, b)(W_{i+1}(a) + W_{i+1}(b)) + \text{mult}_i(g, a, b)(W_{i+1}(a) \cdot W_{i+1}(b))$$

This equation expresses a gate value in layer i using the connected gates in layer $i + 1$



Example

$$a_1 = 2, a_2 = 5, a_3 = 1, a_4 = 3$$

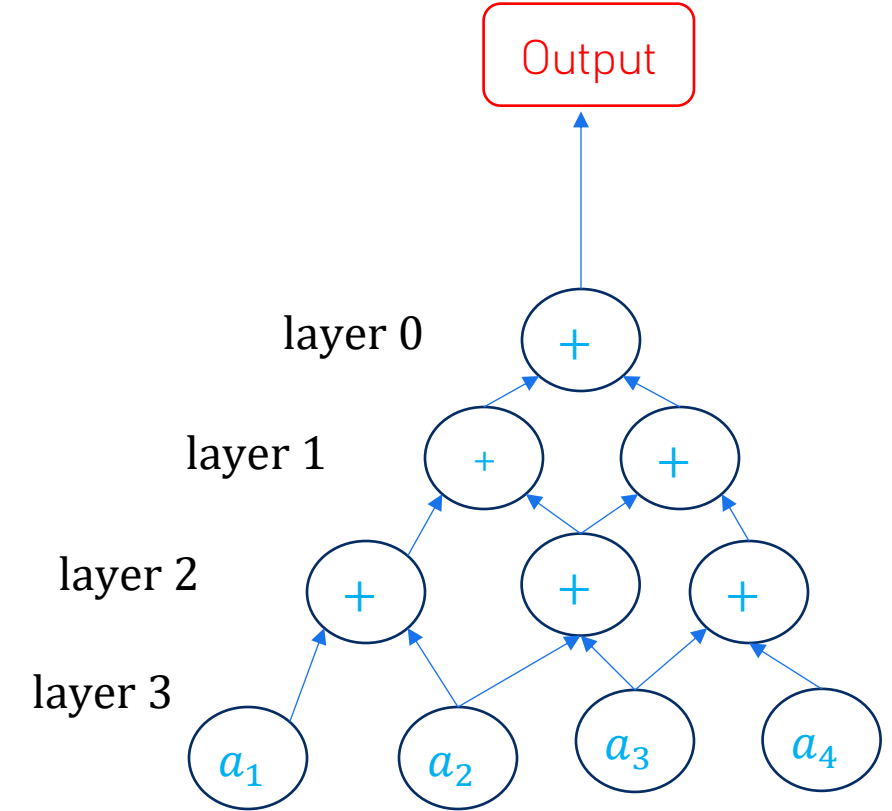
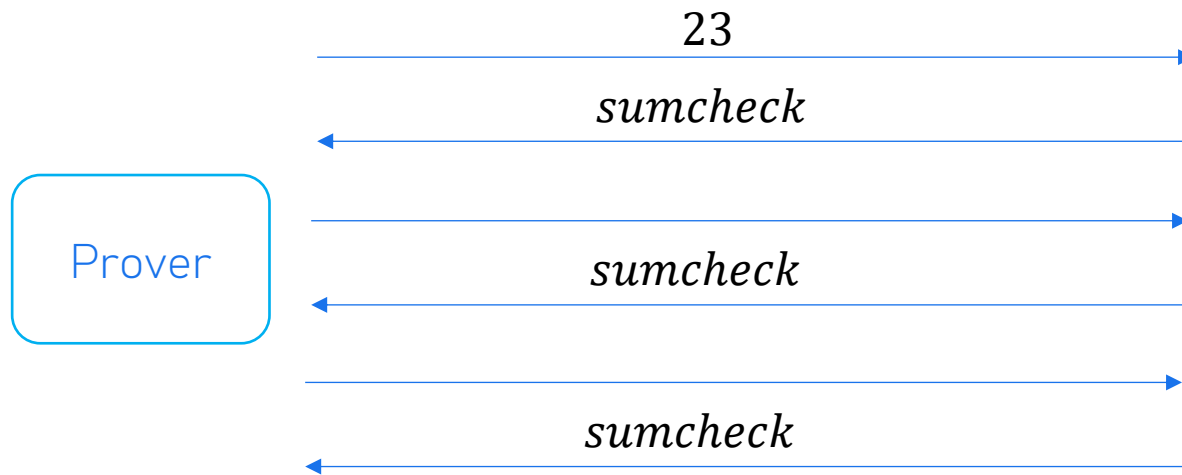
$$w_0 = [23]$$

$$w_1 = [13, 10]$$

$$w_2 = [7, 6, 4]$$

$$w_3 = [2, 5, 1, 3]$$

Claim output will be 23



Verifier

Time Complexity

Number of rounds: d

Prover time: $O(S)$ where S is the total number of gates in the circuit

Verifier time = $O(n + \sum_{i=0}^d \log_2 S_i)$

Approximately this become $O(n + d \cdot \log_2 S)$

Fiat-shamir Transform

Method that turns an interactive proof protocol into non interactive proof protocol

Replaces the verifier's random challenges using a hash function that hashes the transcript so far

$$\begin{aligned} r_i &= H(x, i, r_1, g_1, r_2, g_2, \dots, r_i, g_i) \\ &= H(x, i, r_{i-1}, g_i) \end{aligned}$$

